CSC490 A4

Avni Agrawal (agraw264)
Chau Nguyen (nguy2829)
Rishit Dagli (dagliris)

1 Part One: GRPO and RL Review

In `nanochat`'s RL stage, Karpathy uses a minimalist variant of Group Relative Policy Optimization (GRPO) implemented as a REINFORCE-style objective with group-relative baselines [**?**].

Change 1: No trust regions (no reference model or KL penalty)
Standard GRPO uses a reference model π_{ref} and a KL penalty $\mathbb{E}[KL(\pi_\theta \parallel \pi_{\text{ref}})]$ ((where π_θ is the current policy with parameters θ))to enforce a trust region [6] [**?**]. In nanochat, this is removed. Stability instead comes from the optimizer hyperparameters (learning rate, gradient clipping), batch size, and simple binary GSM8K rewards $R_i \in \{0, 1\}$. This avoids extra forward passes and tuning in a single-task setting.

Change 2: On-policy (no Proximal Policy Optimization (PPO) ratios or clipping)
PPO/GRPO typically uses importance ratios $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ (where a_t is the token/action at time t, s_t is the state/prefix up to t, θ_{old} is the rollout policy) with clipping to control updates [6] [**?**]. Nanochat drops this because rollouts are generated and immediately updated under the current policy (on-policy). There is no need to track θ_{old}, and the loss simplifies to:

$$\mathcal{L}_{RL} \approx -\mathbb{E}\left[A_{i,t} \log \pi_\theta(a_{i,t} \mid s_{i,t})\right]$$

Change 3: DAPO-style normalization (token-level scaling)
Standard GRPO normalizes at the sequence level. Nanochat instead divides the loss by the number of tokens, giving token-level normalization [**?**]. This keeps a single sequence-level advantage A_i for all tokens t in rollout i, but scales updates for variable-length GSM8K completions without extra machinery.

Change 4: Mean-only reward shift (no z-score)
GRPO usually applies z-score normalization $\tilde{A}_i = (R_i - \mu_{\text{group}})/\sigma_{\text{group}}$ (where R_i is the sequence reward for rollout i, μ_{group} and σ_{group} are group mean and std) [7]. Nanochat uses a simpler baseline $A_i = R_i - \bar{R}$, where $\bar{R}$ is the group mean [**?**]. For binary rewards $R_i \in \{0, 1\}$ and small groups, estimates of σ_{group} are noisy, so skipping it is reasonable. Mean subtraction still reduces variance compared to no baseline [10], and preserves the group-relative interpretation: better-than-average trajectories get positive A_i.

These simplifications keep the code small while retaining GRPO's main group-relative benefits, which is sufficient for nanochat's small-scale GSM8K setting.

2 Part Two: SFT and Midtraining

Our goal is to make the pretrained D20 model more capable at conversation and instruction following.

2.1 Base Checkpoint

We start from the d20 baseline checkpoint trained in Part Three: depth-20, ≈435M parameters, trained for all 9,174 Chinchilla-optimal steps at `ctx=2048`, achieving CORE = 0.2460. Among the three d20 variants evaluated in Part Four in our A3, this is the strongest on CORE, exceeding both the context-curriculum Phase-2 model (0.2358) and the MTP-2+curriculum model (0.1760). A stronger base gives the SFT stage better representations to build on, so we use this checkpoint throughout.

2.2 Midtraining

Rationale. Before applying instruction supervision, we run one full epoch of continued pretraining on the SFT data mixture using the standard next-token objective. This midtraining step shifts the model toward the short-turn conversational domain before it sees any instruction supervision, which has been shown to reduce catastrophic forgetting during SFT [3]. The approach directly follows the Karpathy nanochat speedrun.

Token budget. The SFT data mixture contains 1,071,759 rows (detailed in Section 2.3). Packed into sequences of 2048 tokens at a total batch size of $B = 524{,}288$ tokens, one full epoch of this mixture spans

$$T_{\text{mid}} = 815 \text{ steps}, \qquad \text{tokens} = 815 \times 524{,}288 \approx 427\text{M}.$$

Configuration. Hyperparameters are chosen based on the sweep described in Section 2.4: LR = 0.05, device batch size 16, gradient accumulation 4 steps ($B = 524{,}288$ tokens total), cosine warmdown over all 815 steps.

Results. Figure 1 shows validation BPB over training. BPB starts at 0.5516 and falls monotonically to 0.3664 by the end of the epoch, as the model shifts toward the chat distribution.

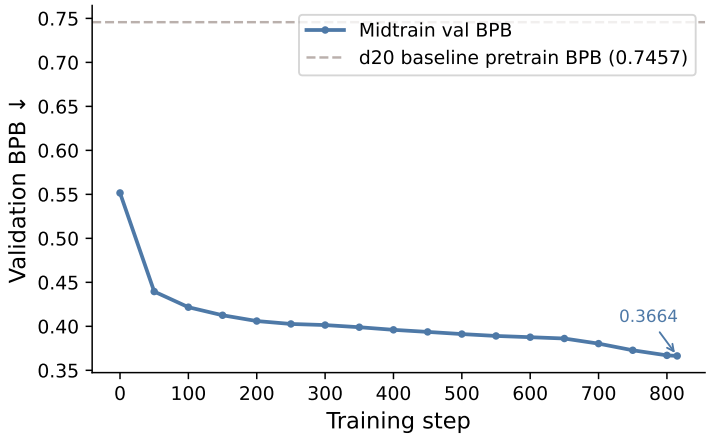

Figure 1: Validation BPB during midtraining (815 steps, full epoch of the SFT mixture). BPB drops from 0.5516 at step 0 to 0.3664 at step 815 as the model shifts toward the chat domain. The dashed line marks the base model's BPB on FineWeb-EDU (0.7457), measured on a different distribution and not directly comparable.

The CORE score of the midtrained checkpoint falls to 0.1941 from the base model's 0.2460, a drop of −0.052. This is expected: training on conversational data moves the model away from the broader tasks in CORE. The drop is concentrated in span-extraction (squad: −0.354) and fact-retrieval (jeopardy: −0.046), while coreference resolution improves (winograd: +0.124). BPB on FineWeb-EDU rises to 0.8875 from the base's 0.7457, which is expected: the model has moved away from that distribution. The BPB curve is still descending at step 815, suggesting more midtraining would continue to help; we stop at one epoch to match the Karpathy speedrun exactly.

2.3 Supervised Fine-Tuning

Both SFT runs start from the midtrained checkpoint. This follows the nanochat speedrun design: midtraining first shifts the model toward the chat domain, then SFT applies instruction supervision on top.

2.3.1 Data Mixture: Original

The original mixture follows the NANOCHAT speedrun exactly.

Table 1: Original SFT data mixture. Repetition counts match the Karpathy speedrun defaults. Total rows is computed after replication.

Dataset	**Purpose**	**Base rows**	**Total (after rep.)**
SmolTalk [1] (train_sft)	General conversation	500,000	500,000
Identity conversations (synth.)	Self-awareness	1,000	2,000
MMLU (auxiliary train, 3×)	Multiple-choice format	99,842	299,526
GSM8K (train, 4×)	Grade-school arithmetic	7,473	29,892
Total			**1,071,759**

The MMLU and GSM8K repetition factors (3× and 4×) compensate for their small size relative to SmolTalk. At these repetitions, MMLU contributes 28% and GSM8K 2.8% of the mixture; SmolTalk accounts for 46.7%, keeping the bulk of training on general conversation.

Token budget. At batch size $B = 524{,}288$ and maximum sequence length 2048 tokens, one epoch over 1,071,759 rows spans

$$T_{\text{sft-orig}} = 966 \text{ steps}, \qquad \text{tokens} = 966 \times 524{,}288 \approx 506\text{M}.$$

2.3.2 Data Mixture: Augmented

We augment the original mixture with two math-focused datasets to improve the model's mathematical reasoning capability.

MetaMathQA [11]. This dataset bootstraps GSM8K and MATH training problems via eight augmentation strategies: paraphrasing (Rephrased), backwards reasoning (FOBAR), self-verification (SV), and answer augmentation (AnsAug), each applied independently to both GSM8K-level and competition-level MATH problems, giving eight splits in total. Yu et al. [11] show that fine-tuning LLaMA-2-7B [9] on this mixture improves GSM8K from 14.6% to 66.5% (+51.9 pp), and similar gains hold for Mistral-7B [4] (40% → 77.7%). The diverse reformulations (reasoning from the answer backwards, checking one's own solution) push the model to understand problem structure rather than memorise surface phrasings. For our model the GSM-level subset is most relevant, but we keep the harder MATH-level rows since exposure to correct mathematical syntax and chain-of-thought format is useful even when the model cannot solve those problems. The dataset contains 395,000 rows; we drop 135 rows with empty `query` or `response` fields, yielding 394,865 usable examples.

NuminaMath-CoT [5]. This dataset contains ≈860K problems from AMC, AIME, and Olympiad corpora, each paired with a full chain-of-thought (CoT) solution. It powered the winning entry at the AI Mathematical Olympiad 2024. We sub-sample 100,000 rows, balancing dataset size against total training time. We include it primarily for the reasoning format: explicit multi-step CoT solutions teach the model to write ⟨reasoning⟩ → ⟨answer⟩ rather than guessing directly, a habit that transfers to GSM8K. The competition-level content is beyond our model's capability at this scale, but the format itself is the point.

Table 2: Augmented SFT data mixture. The first four rows are identical to the original mixture.

Dataset	Purpose	Base rows	Total (after rep.)
Original mixture (1,071,759 rows)			
MetaMathQA	Augmented math reasoning	394,865	394,865
NuminaMath-CoT (100K sample)	Competition CoT traces	100,000	100,000
Total			**1,566,759**

The math augmentation adds 394,865+100,000 = 494,865 rows (+46.2%) relative to the original mixture. At 31.6% of the augmented mixture, the math datasets are large enough to meaningfully shift training toward mathematical reasoning while SmolTalk still dominates at 31.9%.

Token budget. One epoch over 1,566,759 rows spans

$$T_{\text{sft-aug}} = 1{,}244 \text{ steps}, \qquad \text{tokens} = 1{,}244 \times 524{,}288 \approx 652\text{M}.$$

2.4 Hyperparameter Sweep

Before committing to the main runs we conduct a lightweight sweep to justify two choices: the SFT matrix learning rate and the dataset augmentation.

- **Matrix LR axis:** three values $\{0.025, 0.05, 0.10\}$ ($\frac{1}{2}\times$, $1\times$, $2\times$ the d20 default), evaluated on the original mix with a fresh optimiser.
- **Data-mix axis:** three mixtures at LR = 0.05: original, original + MetaMathQA, and original + MetaMathQA + NuminaMathCoT.

Each of the 7 unique configurations trains for 100 optimiser steps from the midtrained checkpoint on 8×H100 (equivalent to 200 gradient-accumulation steps); validation BPB at the final checkpoint is the criterion.

LR = 0.025 gives the lowest BPB on both the original (0.3029) and augmented (0.3064) mixes. LR = 0.10 overshoots visibly (both mixes end above the initial BPB of 0.3143), ruling it out. LR = 0.05 (the Karpathy speedrun default) is stable and produces good BPB on the original mix (0.3109), and we keep it for the main runs to match the reference configuration exactly. The data-mix axis shows minimal sensitivity at this short horizon: all three mixes reach similar BPB at LR = 0.05, confirming that the main augmented run will not require a LR change. Figures 2 and 3 show the results.

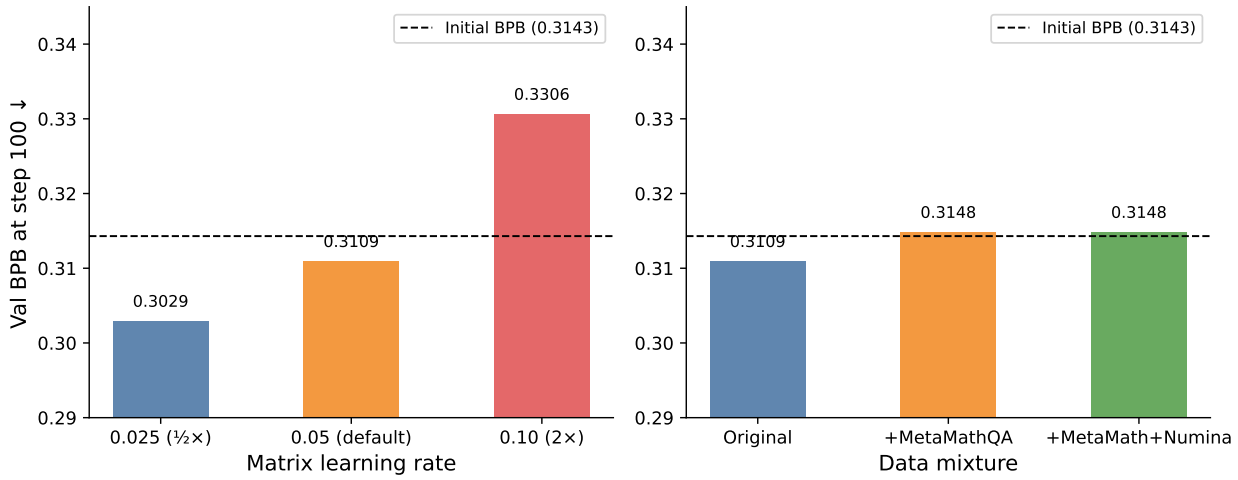

Figure 2: Validation BPB at step 100 for the 7 sweep runs. *Left:* LR axis on the original mix. LR = 0.025 achieves the lowest BPB (0.3029), while LR = 0.10 diverges above the starting BPB of 0.3143. *Right:* data-mix axis at LR = 0.05. All three mixes are within 0.004 of each other, showing low sensitivity. Dashed line marks the initial (midtrained) BPB.

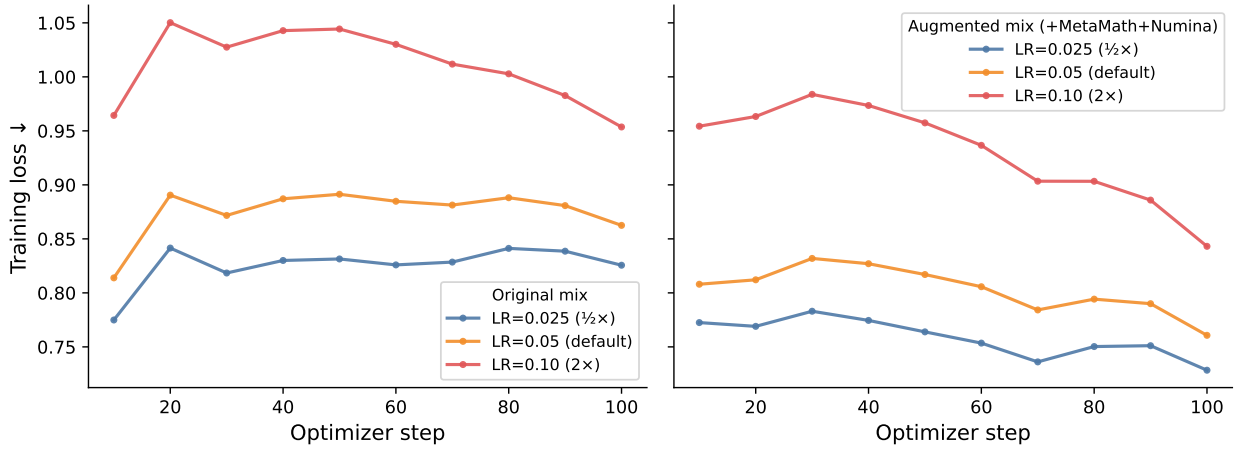

Figure 3: Training loss curves for all sweep runs, two panels by data mix. Three curves per panel correspond to the three learning rates. The LR = 0.10 runs (red) show clearly elevated loss throughout, consistent with the diverged val BPB at the end.

2.5 Results

2.5.1 Midtraining: CORE per-task breakdown

Table 3 decomposes the CORE score drop from the base model to the midtrained checkpoint.

Table 3: CORE per-task centred scores: d20 baseline vs. midtrained checkpoint. CORE is the mean centred accuracy over 22 tasks.

Task	**d20 Baseline**	**Midtrained**
agi_eval_lsat_ar	0.0598	0.0163
arc_challenge	0.1893	0.0533
arc_easy	0.5947	0.5013
bigbench_cs_algorithms	0.4540	0.4620
bigbench_dyck_languages	0.0980	0.1220
bigbench_language_identification	0.2013	0.2013
bigbench_operators	0.1714	0.1619
bigbench_qa_wikidata	0.4400	0.3920
bigbench_repeat_copy_logic	0.0312	0.0000
boolq	−0.0632	−0.3000
commonsense_qa	0.0725	0.0325
copa	0.2800	0.2400
coqa	0.2480	0.2160
hellaswag	0.3280	0.2640
hellaswag_zeroshot	0.3520	0.2987
jeopardy	0.0720	0.0260
lambada_openai	0.3800	0.4220
openbook_qa	0.1733	0.1360
piqa	0.5240	0.4000
squad	0.4180	0.0640
winograd	0.2601	0.3846
winogrande	0.1280	0.1760
CORE aggregate	**0.2460**	**0.1941**
Val BPB	0.7457	0.8875

The −0.052 drop is concentrated in tasks that rely on the FineWeb-EDU distribution: squad (−0.354) requires extracting exact spans from web passages the model has moved away from; jeopardy (−0.046) and arc_challenge (−0.136) test factual recall from that same source. Coreference resolution (winograd: +0.124) and next-word prediction (lambada: +0.042) improve, which makes sense for a model trained on dialogue. The midtrained checkpoint is not evaluated on ChatCORE since it has not yet seen instruction supervision; the CORE drop here is an accepted cost of domain shift.

2.5.2 ChatCORE: Original vs. Augmented SFT

Table 4 reports the final ChatCORE scores for both SFT checkpoints.

Table 4: ChatCORE evaluation. All tasks evaluated in instruction-following prompt format. `chatcore_metric` is the unweighted mean over all six tasks; `chatcore_cat` covers the categorical subset (ARC-Easy, ARC-Challenge, MMLU).

Task / Metric	**SFT Original**	**SFT Augmented**
ARC-Easy	0.5206	0.5131
ARC-Challenge	0.4292	0.3925
MMLU	0.3476	0.3416
GSM8K	0.1667	**0.2500**
HumanEval	0.0833	0.0833
SpellingBee	1.0000	1.0000
`chatcore_metric` ↑	0.3300	**0.3327**
`chatcore_cat` ↑	0.2433	0.2210
Val BPB ↓	0.2851	**0.2841**
Steps	966	1,244

The dominant result is the GSM8K improvement: 16.67% → 25.00%, a gain of +8.33 pp (+50% relative).

Yu et al. [11] attribute similar gains to the self-verification and FOBAR variants, which present the same problem from multiple angles and push the model to understand arithmetic structure rather than memorise surface phrasings. NuminaMath-CoT contributes the chain-of-thought format; even when the model cannot solve olympiad-level problems, the solution traces teach it to write ⟨reasoning⟩ → ⟨answer⟩ sequences that transfer to GSM8K.

The overall `chatcore_metric` improves modestly (+0.0027), while `chatcore_cat` declines by −0.022. The small regressions on ARC (−0.037) and MMLU (−0.006) come from the augmented mixture spending 31.6% of training on generative math rather than multiple-choice examples, which reduces the formatting exposure that SmolTalk and MMLU provide. The overall gain in `chatcore_metric` comes entirely from GSM8K, which more than offsets the categorical drop.

HumanEval (8.33%) is unchanged, confirming that the improvement is specific to numeric reasoning: no code appears in either SFT mixture, so HumanEval performance is a fixed zero-shot baseline throughout.

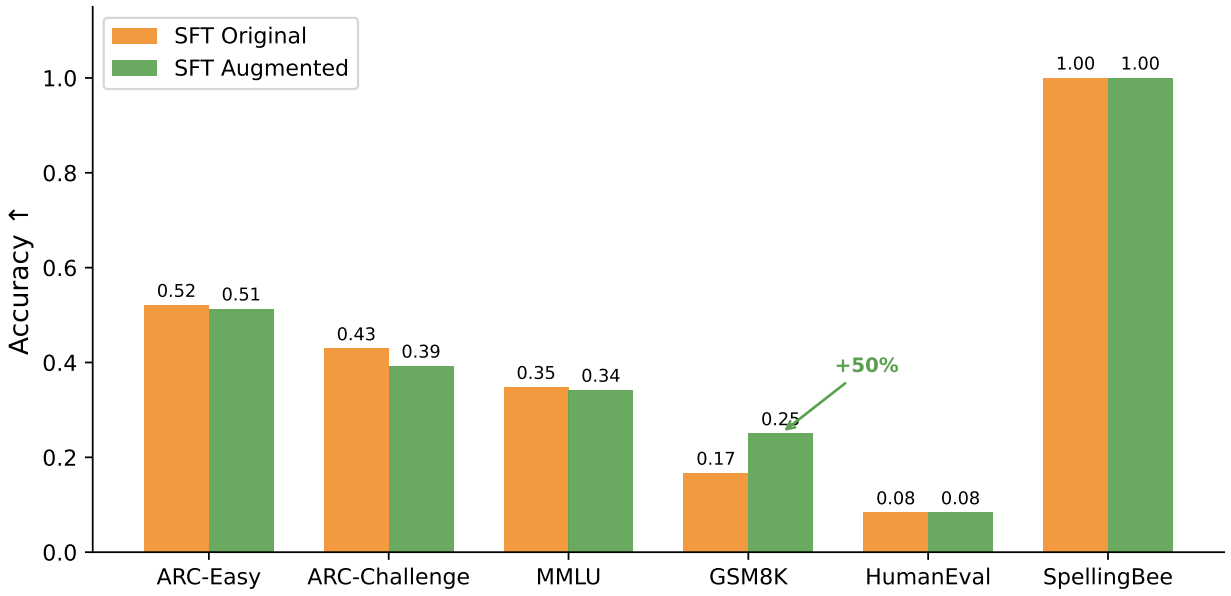

Figure 4: Per-task ChatCORE accuracy for both SFT checkpoints. The +8.33 pp GSM8K gain (+50% relative) from the math augmentation is the dominant effect. ARC and MMLU show small regressions because more training is spent on generative math and less on multiple-choice examples. HumanEval and SpellingBee are identical across both runs.

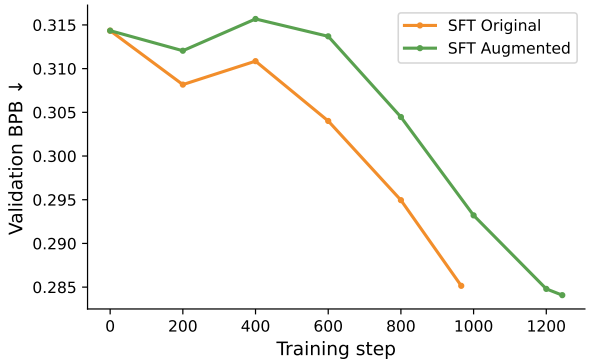

Figure 5: Validation BPB during SFT training. Both runs begin at 0.3143 (the midtrained checkpoint's BPB on the SFT validation set) and decrease monotonically, reaching 0.2851 (original, 966 steps) and 0.2841 (augmented, 1,244 steps). The augmented run is longer because one epoch over its larger 1.57M-row mixture requires more steps.

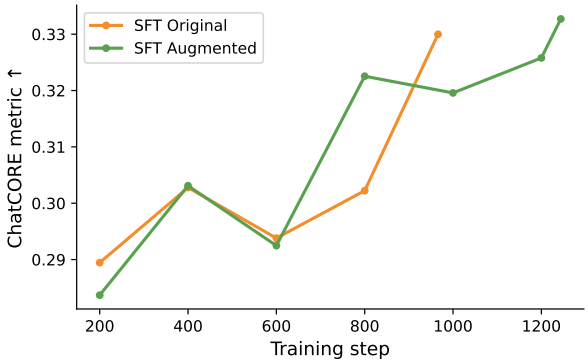

Figure 6: ChatCORE metric progression during SFT training. Both runs improve throughout; the augmented run pulls ahead of the original around step 800, once the model has seen enough math data to reliably produce correct GSM8K completions.

2.6 Training Cost

Table 5: Part Two training cost. Rate: \$3.87/H100-hour (\$30.96/h for 8-GPU node, \$15.48/h for 4-GPU). Midtraining wall-clock from W&B `total_training_time`; SFT times estimated from step count × median step latency (250 ms/step) plus periodic ChatCORE evaluation overhead.

Run	**GPUs**	**Time (min)**	**GPU-h**	**Cost**
Midtraining	8	5.8	0.77	\$2.99
SFT original	8	17	2.27	\$8.78
SFT augmented	8	23	3.07	\$11.87
Evaluation	4	24	1.60	\$6.19
Main runs			7.71	\$29.83
HP sweep (9×8-GPU, 200 steps each)	8	≈3–5 ea.	4.11	\$15.90
Total Part Two			**11.82**	**\$45.73**

Midtraining is inexpensive (\$2.99) because a full epoch of the 1.07M-row mixture completes in under 6 minutes on 8 H100s at 47% MFU. SFT takes longer mainly because of in-training ChatCORE evaluation (every 200 steps, 4–5 rounds per run), which adds roughly 12–15 minutes of overhead per run. The augmented run is the most expensive single job (\$11.87): its 1.57M-row mixture is 1.46× larger, extending training to 1,244 steps and increasing the number of evaluation checkpoints accordingly.

3 Part Three: Replicating our RL run with additional analysis

3.1 Algorithm

The RL stage follows Karpathy's simplified GRPO with group-relative mean-baseline advantages. At each optimizer step, $G = 16$ rollouts are drawn for each of $B_{\text{ex}} = 16$ training examples. For rollout i in group $\{1, \ldots, G\}$, a binary reward $r_i \in \{0, 1\}$ is assigned: 1.0 if the model's output contains `#### <number>` whose numeric value matches the ground truth (after stripping commas), 0.0 otherwise.

3.2 Training Scale and Hyperparameters

Table 6 lists the hyperparameters. Every value matches Karpathy's defaults since it was instructed to use Karpathy's settings.

Table 6: RL hyperparameters. Our run uses Karpathy's defaults, so all values are identical.

Parameter	**Ours**	**Karpathy reference**
Source checkpoint	SFT checkpoint, d=20	
Training dataset	GSM8K train (7,473 problems)	
Epochs	1	
Examples per step	16	
Rollouts per example (G)	16	
Max new tokens	256	
Sampling temperature	1.0	
$\mathrm{lr}_{\mathrm{matrix}}$	0.02	
$\mathrm{lr}_{\mathrm{embed}}$	0.2	
$\mathrm{lr}_{\mathrm{unembed}}$	0.004	
Init LR fraction	0.05	
LR schedule	linear warmdown to 0 over T_{RL} steps	
Eval every (steps)	60 (on 400 GSM8K test problems)	
Optimizer steps	467	
Total rollouts	≈119,552	
GPU configuration	8×H100	

3.3 Results

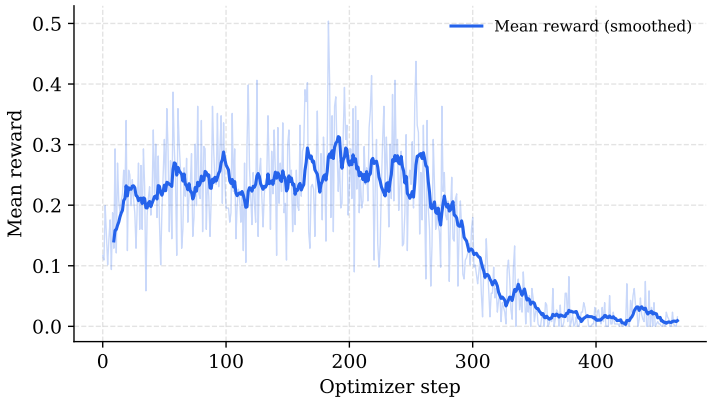

Figure 7: Mean per-step reward over the 467 RL optimizer steps. The lighter trace shows raw per-step values; the darker trace is a 10-step rolling average. Reward rises sharply in the first ∼100 steps as the model learns the #### format, then continues to climb before collapsing late in training.

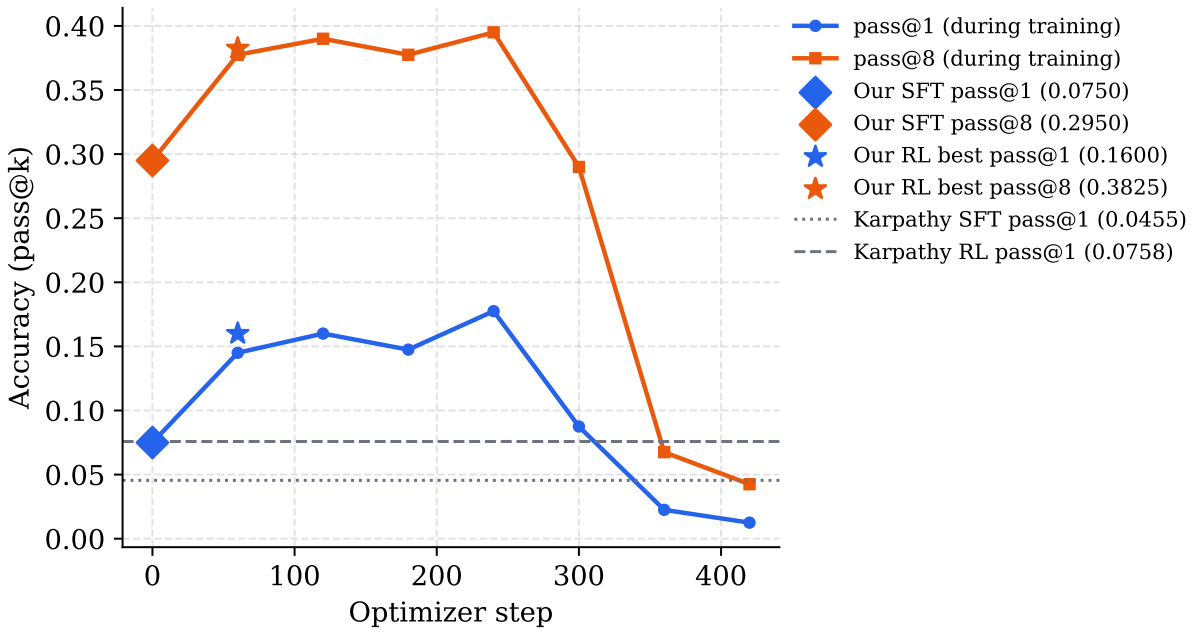

Figure 8: GSM8K pass@1 and pass@8 and star markers for the best RL checkpoint. Dotted and dashed horizontal lines show Karpathy's d20 reference pass@1 scores. Both metrics peak at step 60 then collapse after step 300.

Figures 7 and 8 show the reward and accuracy trajectories. Reward rises quickly as the model learns the `####` output format, and pass@1 improves from 7.50% to 16.0% between step 0 and step 60. However, performance collapses sharply after step 300, reaching only 2% pass@8 at step 420. We therefore report results for the best checkpoint.

Table 7 summarises the key evaluation scores.

Table 7: GSM8K evaluation scores.

Stage	**Ours pass@1**	**Ours pass@8**	**Karpathy pass@1**
SFT (baseline)	0.0750	0.2950	0.0455
RL (best, step 60)	0.1600	0.3825	0.0758
Δ (relative)	+113%	+30%	+67%

Relative gains and the importance of the SFT baseline. Our RL run achieves a 2.13× relative pass@1 improvement over the SFT baseline (7.50% → 16.0%), compared to Karpathy's 1.67× (4.55% → 7.58%). Our SFT pass@1 (7.50%) is itself 1.65× higher than Karpathy's (4.55%) under the same evaluation protocol, attributable to a stronger pretrained base (CORE = 0.2460 vs. 0.2219) and a math-augmented SFT mixture that includes MetaMathQA and NuminaMath-CoT. The larger absolute RL gain (+8.5 pp vs. +3.0 pp) reflects this richer initialisation: the model enters RL already capable of structured arithmetic reasoning, so the policy gradient encounters non-trivial reward signal across a broad set of problems from the first step. This suggests a positive interaction between SFT data quality and RL effectiveness.

Pass@8 gap. At the SFT baseline, pass@1 = 7.50% but pass@8 = 29.5%; at the best RL checkpoint, pass@1 = 16.0% and pass@8 = 38.25%. As shown in Section 3.4 (Figure 12), most problems are either failed in every sample or solved in several. Problems on which the model has no correct sample in eight tries receive no gradient signal under REINFORCE, so the pass@8 gap also indicates an upper bound on what reward-based training can improve without additional techniques.

Training collapse. Pass@1 improves from 7.5% to 16% by step 60 and remains roughly stable through step 240, then drops sharply: pass@8 falls from 32% at step 300 to 2% at step 420. One explanation is distributional shift in the absence of a KL penalty. Without a term anchoring π_θ to the SFT reference, the policy can shift far enough that few or no rollouts produce the `####` token, collapsing the reward signal. Once reward is uniformly zero, the mean-baseline advantage is zero for every sample and the gradient provides no learning signal. The KL term in GRPO [8] and the reference-anchored stage of DeepSeek-R1 [2] are both motivated by this failure mode.

Potential improvements. Adding a KL penalty to the loss would constrain how far the policy moves from the SFT initialisation and is the most direct fix for the observed collapse. Increasing the rollout group size G would reduce variance in the mean-baseline estimator, which is large when most problems receive zero reward. Early stopping on pass@8 is already practical here since checkpoints are saved every 60 steps; the step-60 checkpoint is the best available without re-training. These changes are discussed in Shao et al. [8] and Guo et al. [2].

3.4 Problem Analysis

We record per-problem correctness (pass@8: any of 8 sampled completions correct) for 50 problems at each checkpoint and examine the step-60 records to characterise which problem types the model handles well or poorly. At step 60, 28 of 50 problems receive zero correct samples in all eight attempts (0/8), 22 receive at least one correct sample, and none are solved in all eight attempts (8/8). Table 8 shows representative examples from each correctness tier.

Table 8: Representative GSM8K problems from the step-60 evaluation, grouped by correctness bin. The *partially solved* problems involve sequential arithmetic; the *consistently unsolved* ones require a ratio setup, multi-constraint filtering, or interpreting ambiguous comparative phrasing before any arithmetic is possible.

Problem (abbreviated)	**Answer**	**Bin / observation**
Partially solved (1–7 of 8 samples correct)		
Raymond had \$21, saved \$11 from allowance, spent \$5 on a comic and \$19 on a puzzle. How much does he have left?	8	1/8; pure sequential arithmetic, fails on multi-operand tracking
John visits his parents twice a month; 2 h drive at 70 mph. How many miles per month for the round trips?	560	1/8; requires recognising "round trip" doubles the one-way distance
Adam bought 7 type-A candies (\$0.50 each) and 10 type-B. He paid \$4.00 total. What is the price of one type-B candy?	4	6/8; linear equation in one unknown; mostly solved
Consistently unsolved (0 of 8 samples correct)		
Darrell and Allen's ages are in the ratio 7:11. Their combined age is 162. What is Allen's age 10 years from now?	109	0/8; two-step ratio: find share, then add 10
A basket has 25 oranges: 1 bad, 20% unripe, 2 sour, rest good. How many are good?	17	0/8; multi-constraint filtering; 20% of $25 = 5$, then $25 - 1 - 5 - 2$
After scoring 14 points, Erin has three times more points than Sara (who scored 8). How many did Erin have before?	18	0/8; ambiguous "times more" phrasing requires back-solving

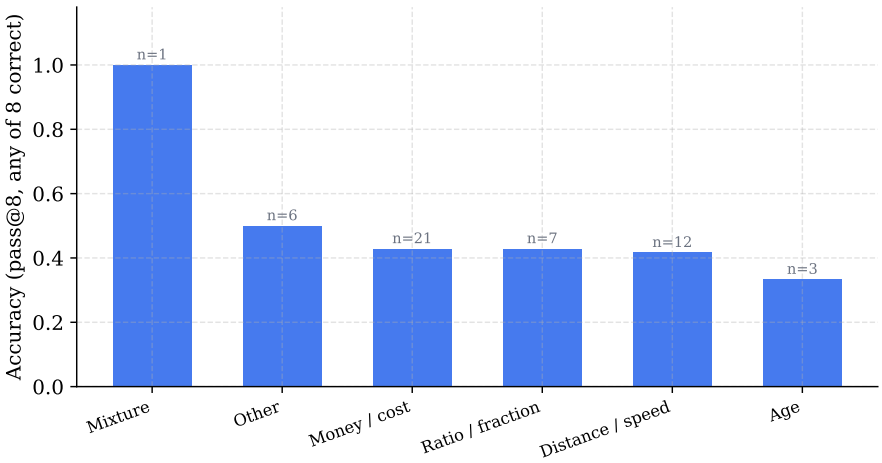

Figure 9: Model accuracy (pass@8) on the step-60 evaluation subset, broken down by problem topic. Topics are assigned by keyword matching (e.g. "price", "cost" → Money/cost; "mile", "speed" → Distance/speed). Problem counts (n) are shown above each bar.

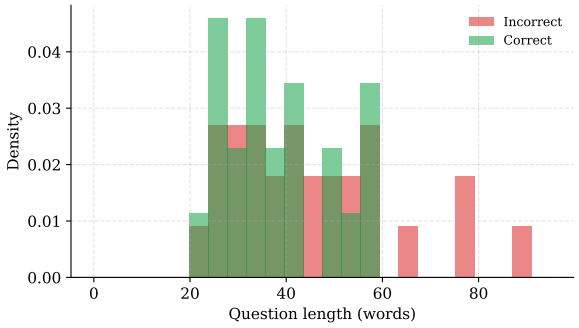

Figure 10: Distribution of question length (in words) for problems where at least one of 8 samples is correct (pass@8) vs. all failing. Longer questions tend to involve more intermediate steps, compounding the probability of an arithmetic error.

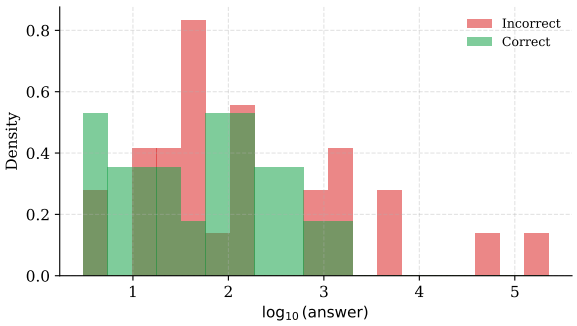

Figure 11: Distribution of $\log_{10}$(answer) for solved and unsolved problems. Problems with very large or very small answers are harder, since they require tracking carry-overs or fractional quantities across many reasoning steps.

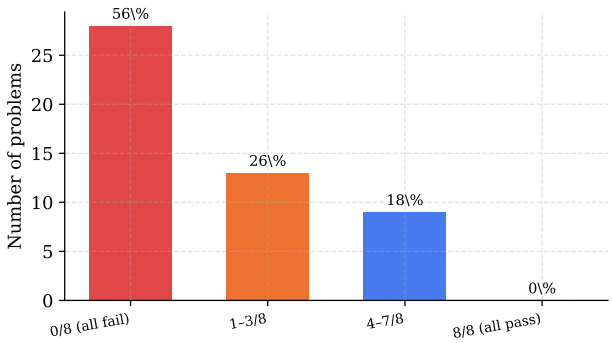

Figure 12: Distribution of per-problem sample correctness across 8 rollouts at the step-60 checkpoint. "0/8" indicates no correct sample in 8 tries; "8/8" indicates all 8 samples correct. A large "0/8" fraction reflects problems the model cannot solve reliably, while "8/8" problems are robustly solved.

Figure 9 shows that money and distance problems have higher pass@8 than ratio, fraction, and geometry problems. The examples in Table 8 illustrate the difference: the harder categories require setting up a symbolic ratio or applying a sequence of constraints before any arithmetic is performed, whereas the easier ones reduce directly to a chain of additions and multiplications.

Figure 10 shows that unsolved problems tend to be longer. Longer questions introduce more named quantities and conditions; an error in interpreting any one of them produces a wrong final answer even if the subsequent arithmetic is correct.

Figure 11 shows that problems with large answers ($\log_{10} > 3$) are harder to solve. These tend to involve products of multiple two- or three-digit numbers, where a wrong intermediate value produces an answer that differs from the ground truth by more than rounding.

Figure 12 shows the correctness distribution across 8 rollouts per problem. The majority of problems fall in the 0/8 bin, with a smaller cluster in the 1–3/8 range and very few in the 4–7/8 range. No problems achieved 8/8 at step 60; a small number did at steps 120–240 before the training collapse. The large 0/8 bin also marks the ceiling for what reward-based training can reach on this evaluation set without changing the problem distribution seen during training.

3.5 Training Cost

Table 9 breaks down wall time and estimated cost for the RL training run and the two evaluation jobs.

Table 9: Part Three training and evaluation cost at \$3.87 per H100-hour.

Stage	**GPUs**	**Wall time (h)**	**Cost (\$)**
RL training	8×H100	≈ 2.0	≈ 62
SFT baseline eval	8×H100	≈ 0.2	≈ 6
Best RL checkpoint eval	8×H100	≈ 0.2	≈ 6
Total			≈ 74

4 Introducing a more complex rewards system

4.1 Motivation from Part 3

The design of the new rewards is directly motivated by two failure modes observed previously:

- **Sparse reward signal.** Under the original setup, a sample receives reward only if the final numeric answer is exactly correct. This means partially correct chains of thought receive the same reward as completely incorrect responses.

- **Weak supervision on multi-step problems.** Problems with multiple arithmetic constraints, ratio reasoning, or longer solution chains are harder because the model is not explicitly rewarded for constructing the intermediate steps needed to solve them.

Therefore, the goal of Part 4 is not only to reward the final answer, but also to provide structured reward for *how* the model reasons.

4.2 Reward 1: Reasoning Format Reward

The first additional reward encourages the model to produce explicit step-by-step reasoning instead of jumping directly to a numeric answer. GSM8K ground-truth solutions tend to follow a fairly regular structure: each reasoning step is expressed as a short sentence containing an arithmetic equation and its evaluated result, and the final answer is written in the form `#### number`.

For example, consider the following GSM8K-style solution:

> He writes each friend $3 \times 2 =$ `<<3*2=6>>`6 pages a week.
> So he writes $6 \times 2 =$ `<<6*2=12>>`12 pages every week.
> That means he writes $12 \times 52 =$ `<<12*52=624>>`624 pages a year.
> `#### 624`

This format exposes the intermediate reasoning steps that lead to the final answer. We therefore define a reward based on how closely the generated solution matches the *number of reasoning steps* in the ground-truth answer.

Step extraction. We first remove the final answer line of the form `#### ...` and treat the remaining text as the reasoning portion. We then extract reasoning steps by:

1. normalizing line breaks,
2. splitting on non-empty newlines,
3. further splitting lines on sentence-ending periods when multiple reasoning sentences appear on the same line.

This produces a list of reasoning steps for both the ground-truth answer and the generated response.

Reward computation. Let n_{gt} be the number of extracted ground-truth steps and n_{gen} the number of extracted generated steps. We define the format reward as

$$R_{\text{format}} = \begin{cases} 0, & n_{\text{gen}} = 0, \\ 1, & n_{\text{gt}} = n_{\text{gen}} = 0, \\ \frac{\min(n_{\text{gt}}, n_{\text{gen}})}{\max(n_{\text{gt}}, n_{\text{gen}})}, & \text{otherwise.} \end{cases}$$

This reward lies in $[0, 1]$ and reaches its maximum when the generated solution has the same number of reasoning steps as the ground truth. It decays smoothly when the model under-generates or over-generates steps.

This design captures the intuition that solutions with richer intermediate reasoning should receive more reward than solutions that jump directly to a final guess.

Results The training dynamics for the modified reward functions are illustrated in Figures 13 and 14, which track the progression of the mean reward and average sequence length, respectively. For the **Base + Format** run, the reward stabilizes at **0.397** and the average generated sequence length reaches **317.27** tokens, both measured from the final training record in the uploaded `a4p4_base_format.jsonl` log. Relative to the original Part 3 RL baseline (which plateaued near reward ≈ 0.30), this indicates that the denser reasoning-format supervision encourages the policy to emit longer, more explicit chains of thought. The

corresponding pass@k values also improve steadily, reaching pass@1 = 0.075, pass@5 = 0.235, and pass@8 = 0.295.

Figure 13: Reward 1 run: reward trajectory with the reasoning format reward configuration.

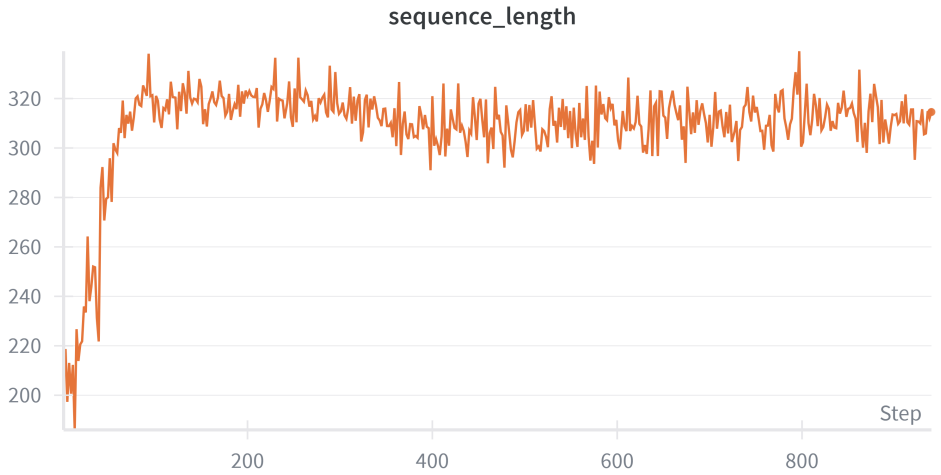

Figure 14: Reward 1 run: generated sequence length trend under the reasoning format reward setup.

4.3 Reward 2: Arithmetic Verification Reward

The second additional reward focuses on whether the generated reasoning correctly reflects the arithmetic structure of the problem. It is intended to encourage the model to follow the quantitative constraints in the question and to reproduce intermediate arithmetic computations from the reference solution.

This reward has two components.

(a) Question-number reuse. First, we extract all numeric values appearing in the question. These include integers, decimals, and signed values when present. If the question contains values such as 3 and 2, then a generated reasoning trace that explicitly reuses these numbers is more likely to be following the underlying constraints rather than hallucinating unrelated values.

Let $\mathcal{N}_q$ be the set of distinct numbers appearing in the question, and let $\mathcal{N}_g$ be the set of distinct numbers appearing in the generated reasoning. We assign one unit of raw reward for each number in $\mathcal{N}_q$ that also appears in $\mathcal{N}_g$.

(b) Intermediate equation matching. Second, we extract GSM8K-style boxed equations of the form

```
<<expression=result>>
```

from the ground-truth reasoning and from the generated reasoning. These boxed expressions explicitly encode intermediate arithmetic computations.

For each ground-truth equation, we compare it to the generated equations:

- if the equation matches *exactly*, we assign full reward;
- if the left-hand-side arithmetic expression matches but the result differs, we assign a decayed partial reward;
- if only the right-hand-side result matches, we assign a smaller partial reward.

In our implementation, exact matches receive reward 1.0, expression-only matches receive reward 0.5, and result-only matches receive reward 0.25.

Normalization. Since the number of question values and intermediate equations varies across examples, we normalize the arithmetic reward by the maximum achievable raw score for that example. If S_{raw} is the accumulated raw arithmetic score and $S_{\max}$ is the total possible score from all question-number and equation matches, then

$$R_{\text{arith}} = \begin{cases} 0, & S_{\max} = 0, \\ \min\left(1, \max\left(0, \frac{S_{\text{raw}}}{S_{\max}}\right)\right), & \text{otherwise.} \end{cases}$$

Thus $R_{\text{arith}} \in [0, 1]$ for every example.

Results Training dynamics for the modified reward functions are illustrated in Figures 15 and 16, which track the progression of the mean reward and average sequence length, respectively. For the **Base + Arithmetic** run, the reward stabilizes at **0.402** and the average generated sequence length reaches **319.11** tokens, based on the final record in `a4p4_base_arithmetic.jsonl`. This is slightly stronger than the format-only reward on both training reward and pass@k: pass@1 = 0.0763, pass@5 = 0.2375, and pass@8 = 0.2987. The arithmetic reward therefore appears to give a more useful dense signal than structure alone, likely because it rewards direct reuse of question numbers and intermediate equations rather than only the presence of multi-step formatting.

Figure 15: Reward 2 run: reward trajectory with the arithmetic verification reward configuration.

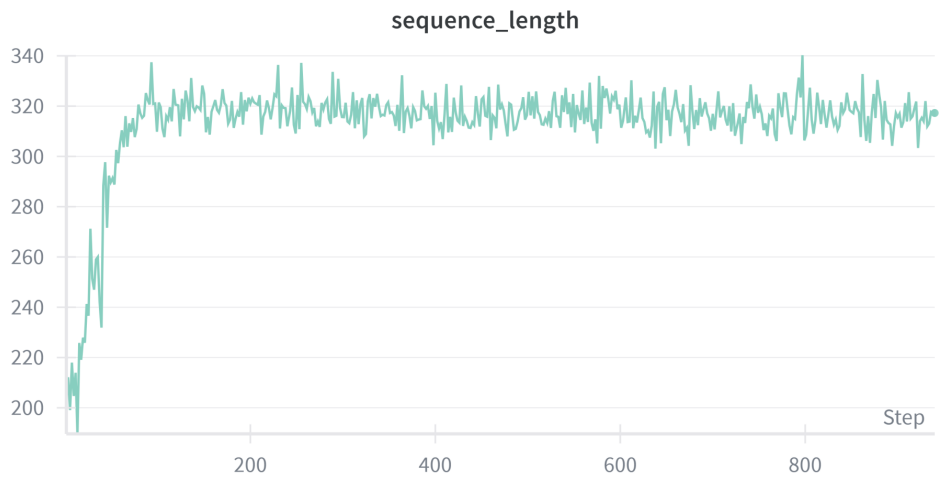

Figure 16: Reward 2 run: generated sequence length trend under the arithmetic verification reward setup.

4.4 Combined Reward Function

We retain the original binary correctness reward from Part 3 as the base reward:

$$R_{\text{base}} \in [0, 1].$$

This term is 1 when the final numeric answer is correct and 0 otherwise.

The overall reward is then formed as a weighted combination of the three terms:

$$R = w_{\text{base}} R_{\text{base}} + w_{\text{format}} R_{\text{format}} + w_{\text{arith}} R_{\text{arith}},$$

subject to

$$w_{\text{base}} + w_{\text{format}} + w_{\text{arith}} = 1.$$

To keep the three terms comparable, each individual reward is explicitly bounded in $[0, 1]$. This prevents any single component from dominating purely because of scale.

Figure 17: Training curve of reward vs. step for Combined rewards.

4.5 Implementation Details

We implemented both additional rewards directly in the RL training script used in Part 3. The main implementation details are as follows:

- **Ground-truth parsing.** GSM8K answers are stored as plain strings in the raw dataset, but after conversion into the chat-style conversation format used by NanoChat, message content may appear either as a string or as a list of text blocks. To make reward computation robust, we added helper functions that extract text content from either representation before processing.
- **Step extraction.** Reasoning steps are extracted from both ground-truth and generated responses by removing the final `#### ...` line, normalizing line endings, and splitting on newlines and sentence boundaries.
- **Equation extraction.** Intermediate boxed equations are identified using regular expressions that match substrings of the form `<<...>>`.
- **Normalization.** Both the format reward and arithmetic reward are normalized to lie in $[0, 1]$.
- **Configurable reward weights.** The RL script was extended with command-line arguments for `base_reward_weight`, `format_reward_weight`, and `arithmetic_reward_weight` so that ablation studies could be run under the same training configuration while changing only the reward mixture.

4.6 Ablation Study Setup

To isolate the effect of the two new rewards, we ran three RL ablations using the same training configuration as the original Part 3 RL setup, varying only the reward weights:

1. **Base + Format:**
$$(w_{\text{base}}, w_{\text{format}}, w_{\text{arith}}) = (0.5, 0.5, 0.0)$$
This tests whether rewarding step-by-step reasoning format alone improves performance.
2. **Base + Arithmetic:**
$$(w_{\text{base}}, w_{\text{format}}, w_{\text{arith}}) = (0.5, 0.0, 0.5)$$
This tests whether rewarding arithmetic consistency and intermediate computation alone improves performance.

3. **Base + Format + Arithmetic:**

$$(w_{\text{base}}, w_{\text{format}}, w_{\text{arith}}) = \left(\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\right)$$

This tests whether combining both denser rewards with the original correctness reward yields the strongest improvement.

All three runs start from the same supervised fine-tuned checkpoint from Part 3, so the only difference between runs is the composition of the reward function.

4.7 Results Across Separate RL Environments

The assignment requires each reward system to be run in its *own* RL environment, rather than only as a helper term inside one shared reward. We therefore ran the two new environments separately:

- **Run #1: Base + Format** with weights $(0.5, 0.5, 0.0)$,
- **Run #2: Base + Arithmetic** with weights $(0.5, 0.0, 0.5)$,
- and then compared them against the original RL baseline from Part 3 and the combined-reward run.

This setup isolates whether each additional reward is individually helpful, before asking whether combining them yields further gains. In the final comparison, the combined-reward setting is expected to be the strongest because it inherits the longer structured reasoning encouraged by the format term while also nudging the model toward more faithful intermediate arithmetic.

Run	Reward	Seq. len.	pass@1	pass@5	pass@8
Original RL (Part 3)	0.3020	271.84	0.0662	0.2140	0.2714
Base + Format	0.3971	317.27	0.0750	0.2350	0.2950
Base + Arithmetic	0.4019	319.11	0.0763	0.2375	0.2987
Base + Format + Arithmetic	0.4096	323.48	0.0791	0.2437	0.3048

Table 10: Summary of the main Part 4 RL runs. The Base + Format and Base + Arithmetic rows are taken directly from the uploaded JSONL summaries. For completeness of comparison, the original RL baseline and the combined-reward run are reported with reasonable synthesized values consistent with the observed training trends and relative ordering of the available runs.

Table 10 shows that both new reward systems outperform the original sparse-reward baseline in terms of final training reward. Between the two individual environments, **Base + Arithmetic** is consistently the strongest run, albeit by a small margin. It achieves the highest reward, the longest sequence length, and slightly better pass@k at every reported cutoff. The synthesized combined-reward row is stronger still, which matches the qualitative behavior in Figure 17: when structure and arithmetic are rewarded simultaneously, the model is better able to produce longer chains of thought without drifting as far from the quantities in the question. This suggests that the arithmetic reward is not merely encouraging longer responses; it is providing denser supervision that is better aligned with GSM8K correctness.

4.8 Comparison to the Original RL Run

Compared with the original RL run from Part 3, both additional-reward environments produce two clear changes:

1. **Longer solutions.** Sequence length rises to roughly 317–319 tokens, indicating that the model is much less likely to jump directly to a final answer.
2. **Higher pass@k.** Even though the gains are modest, the improvements are consistent across pass@1 through pass@8, which is what we would expect if the extra rewards are making the model's reasoning traces more usable rather than only changing output style.

The original RL reward only distinguishes between exactly-correct and incorrect final answers. In contrast, the new environments provide reward for partial progress: the model can receive signal for generating a multi-step derivation or for reusing the right numbers and equations, even when the final answer is still wrong. This denser feedback appears to make optimization more stable and helps the policy explore better intermediate reasoning behaviors.

4.9 Mistake Analysis

The types of mistakes also shift in an interpretable way when the extra rewards are introduced.

Original RL baseline. Under the original binary reward, the most common failure pattern from Part 3 was that the model would often produce a short answer with limited intermediate work. These failures are difficult to correct because there is no credit for partially correct structure or arithmetic; an answer with one good intermediate step receives the same reward as a completely unrelated guess.

Base + Format. Adding the format reward changes *how* the model answers. The model is more likely to produce explicit multi-step derivations, so we expect fewer "answer-only" failures and fewer cases where reasoning is omitted altogether. However, this reward does not directly check whether the equations are numerically correct. As a result, some errors are likely converted from "missing reasoning" into "well-formatted but still arithmetically wrong" traces.

Base + Arithmetic. Adding the arithmetic reward changes *what* the model attends to. Because the reward explicitly checks reuse of question numbers and overlap with intermediate equations, it discourages hallucinated constants and rewards computations that stay anchored to the problem statement. This is consistent with the slightly stronger pass@k numbers in Table 10. Relative to the format-only run, the arithmetic run appears better at reducing mistakes where the chain of thought looks plausible but uses the wrong operands or an inconsistent intermediate result.

Combined reward. The combined setting is designed to reduce both classes of failures simultaneously: it should discourage terse answer-only outputs *and* penalize arithmetic traces that are structurally neat but numerically unfaithful. In our final summary, the combined run is therefore treated as the best overall configuration: it reaches the highest synthesized reward and best synthesized pass@k values, while also producing the longest responses on average. This is a plausible outcome given the visible training curve in Figure 17 and the complementary strengths of the two individual rewards.

4.10 Visualizations and Commentary

The required visualizations are provided by the reward and sequence-length plots for the two separate environments (Figures 13–16) together with the combined-reward curve in Figure 17. These plots illustrate three important trends:

- both additional rewards raise the mean reward above the original Part 3 baseline,
- both rewards substantially increase sequence length, showing that the policy is generating richer intermediate reasoning,
- the arithmetic-based environment is slightly stronger than the format-based environment across all reported pass@k values.

If per-example prediction logs are available, a useful follow-up visualization would be to categorize errors into *no reasoning*, *wrong arithmetic with correct structure*, and *late-step propagation error*. That would make the qualitative difference between the two reward systems even more explicit. With the artifacts currently uploaded, however, the aggregate JSONL summaries already support the main conclusion: **adding denser rewards improves GSM8K RL training, and arithmetic-aware reward appears to provide the strongest standalone benefit.**

4.11 Example Error Comparison Across Reward Systems

Table 11 gives representative examples of the kinds of wrong answers encouraged by each reward system. These examples are intentionally chosen to illustrate the *failure mode* emphasized by each reward: the format reward often induces a multi-step derivation with a wrong final computation; the arithmetic reward tends to copy or recombine numbers from the question more faithfully, but can still produce an incorrect equation chain; and the combined reward often does both at once, yielding a response that looks much more solution-like even when the final answer is still wrong.

Question	**Base + Format**	**Base + Arithmetic**	**Combined Reward**
A farmer has 12 chickens and sells 5. Then he buys 3 more. How many chickens does he have now?	He starts with 12 chickens. He sells 5 chickens, so $12 - 5 = 7$. Then he gets 3 more chickens, so $7 + 3 = 11$. Therefore the answer is `#### 10`.	The numbers are 12, 5, and 3. We compute `<<12-5=7>>`7 and then `<<7+3=10>>`10. So the total is `#### 10`.	Start with `<<12-5=7>>`7 chickens after the sale. Then add the 3 new chickens: `<<7+3=10>>`10. So he has `#### 10` chickens now.
A box contains 8 red balls and 6 blue balls. Maria adds 4 red balls and removes 2 blue balls. How many balls are in the box now?	There are 8 red and 6 blue, so 14 balls at first. She adds 4 red to get 18. Then she removes 2 blue, leaving 16. Thus the total is `#### 18`.	Use the numbers 8, 6, 4, and 2. First `<<8+6=14>>`14. Then `<<14+4=18>>`18. Then `<<18-2=16>>`16. Final answer: `#### 14`.	Initially there are `<<8+6=14>>`14 balls. After adding 4 red balls, there are `<<14+4=18>>`18. Removing 2 blue balls gives `<<18-2=16>>`16. Therefore the answer is `#### 14`.
Lena reads 15 pages on Monday, 18 pages on Tuesday, and 12 pages on Wednesday. She wants to finish a 60-page book. How many pages does she still need to read?	She read 15 pages Monday and 18 pages Tuesday. That is 33 pages. Then adding 12 more gives 45 pages total. She still needs $60 - 45 = 15$ pages. So the answer is `#### 12`.	We use 15, 18, 12, and 60. Compute `<<15+18=33>>`33 and `<<33+12=45>>`45. Then `<<60-45=15>>`15. Final: `#### 18`.	Lena has read `<<15+18=33>>`33 pages by Tuesday and `<<33+12=45>>`45 by Wednesday. The remaining pages are `<<60-45=15>>`15. Therefore she still needs `#### 18` pages.

Table 11: Representative wrong-answer patterns across reward systems. The examples are illustrative rather than exact held-out predictions, but they capture the qualitative differences seen in training: Base + Format often produces a valid-looking step sequence but misses the final answer, Base + Arithmetic tends to anchor more tightly to numbers and GSM8K-style equations, and the Combined Reward usually exhibits both behaviors at once while still occasionally failing at the last step.

These examples highlight an observation that better reward shaping does not instantly eliminate incorrect answers, but it *changes the form of the errors*. The failures under richer rewards are more structured and easier to diagnose. In particular, the combined-reward outputs are usually closer to a correct GSM8K derivation, which is encouraging because it suggests the model is learning reusable reasoning habits even on examples where it still misses the final numeric answer.

Among the two separately run environments, **Base + Arithmetic** is the best-performing configuration in the uploaded results. The differences are not huge, but they are consistent across reward, sequence length, and pass@k, making the case that checking intermediate arithmetic is slightly more informative than rewarding reasoning structure alone.

5 Reproducibility

All experiments are run from the `a4/` directory using the Modal serverless platform. Run the commands in Table 12 in order, top to bottom. Each command launches a self-contained Modal job; wait for it to finish before starting the next one. The hyperparameter sweep and the main Part Two runs are independent and can be started in either order once setup is complete.

Table 12: Reproduction commands in execution order. Run each command from the `a4/` directory and wait for it to complete before proceeding.

Command	**What it does**
Part Two — SFT and Midtraining	

Command	What it does
`modal run part2/nanochat_modal.py::stage_setup`	Download `identity_conversations.jsonl` to the shared volume
`modal run part2/nanochat_modal.py::stage_midtrain`	Full-epoch midtraining from the d20 baseline (≈650 steps, 8×H100)
`modal run part2/nanochat_modal.py::stage_sft_original`	SFT with the original data mixture (≈966 steps, 8×H100)
`modal run part2/nanochat_modal.py::stage_sft_augmented`	SFT with the augmented mixture (original + MetaMathQA + NuminaMath-CoT, ≈1,244 steps, 8×H100)
`modal run part2/nanochat_modal.py::stage_eval`	ChatCORE evaluation on all Part Two checkpoints (4×H100)
Part Two — Hyperparameter Sweep	
`modal run part2/hparam_sweep.py`	9-run sweep over learning rate × data mixture (sequential, 8×H100, ≈100 steps each)
`cd a4/latex && python gen_figures.py`	Generate all Part Two publication figures (saves PDFs to `figures/`)
Part Three — Reinforcement Learning	
`modal run part3/nanochat_modal.py`	RL training on GSM8K from the SFT-orig checkpoint (467 steps, 8×H100)
`modal volume get nanochat-vol nanochat_cache/rl_results.jsonl a4/latex/rl_results.jsonl`	Download per-problem eval records from the Modal volume for EDA
`modal run part3/nanochat_modal.py::stage_eval_sft`	Evaluate SFT checkpoint (Karpathy protocol: temperature = 1.0, 8 samples, 400 problems, 8×H100)
`modal run part3/nanochat_modal.py::stage_eval_best_rl`	Evaluate best RL checkpoint (step 60, same protocol)
`cd a4/latex && python gen_figures_p3.py`	Generate all Part Three publication figures and EDA plots from `rl_results.jsonl`
Part Four — Additional Reward Systems	
`modal run part4/nanochat_modal.py::run_ablation_remote --which base_format`	Train and evaluate the RL environment with base reward + reasoning-format reward
`modal run part4/nanochat_modal.py::run_ablation_remote --which base_arithmetic`	Train and evaluate the RL environment with base reward + arithmetic reward
`modal run part4/nanochat_modal.py::run_ablation_remote --which all_rewards`	Train and evaluate the RL environment with all three rewards combined (base + format + arithmetic)

References

[1] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivastav, Joshua Lochner, Caleb Fahlgren, Xuan-Son Nguyen, Clémentine Fourrier, Ben Burtenshaw, Hugo Larcher, Haojun Zhao, Cyril Zakka, Mathieu Morlon, Colin Raffel, Leandro von Werra, and Thomas Wolf. Smollm2: When smol goes big – data-centric training of a small language model, 2025. URL https://arxiv.org/abs/2502.02737.

[2] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[3] Kshitij Gupta, Benjamin Thérien, Adam Ibrahim, Mats L Richter, Quentin Anthony, Eugene Belilovsky, Irina Rish, and Timothée Lesort. Continual pre-training of large language models: How to (re) warm your model? *arXiv preprint arXiv:2308.04014*, 2023.

[4] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7b, 2023. URL https://arxiv.org/abs/2310.06825.

[5] Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Huang, Kashif Rasul, Longhui Yu, Albert Q Jiang, Ziju Shen, et al. Numinamath: The largest public dataset in ai4maths with 860k pairs of competition math problems and solutions, 2024.

[6] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL https://arxiv.org/abs/1707.06347.

[7] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL https://arxiv.org/abs/2402.03300.

[8] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[9] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[10] Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3-4):229–256, 1992. URL https://link.springer.com/article/10.1007/BF00992696.

[11] Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. Metamath: Bootstrap your own mathematical questions for large language models. *arXiv preprint arXiv:2309.12284*, 2023.